

**Politechnika Wrocławska**

Wydział Elektroniki, Fotoniki i Mikrosystemów

## **SPRAWOZDANIE**

**Projekt sterownika windy  
w języku VHDL**

**Złożono: 26.01.2026r.**

**WROCŁAW 2026**

# 1. Opis projektu

## 1.1 Cel projektu

Celem projektu było zaprojektowanie sterownika windy pięciopiętrowej w języku opisu sprzętu VHDL. Sterownik miał realizować podstawowe funkcje windy pasażerskiej. Projekt zakładał stworzenie systemu sterowania, który będzie w stanie obsługiwać żądania pasażerów, zarządzać ruchem kabiny oraz reagować na sytuacje awaryjne.

Głównym założeniem projektowym była obsługa pięciu pięter, obejmujących parter oznaczony jako piętro zerowe oraz piętra od pierwszego do czwartego. Sterownik miał umożliwiać realizację jazdy zarówno w górę, jak i w dół, z możliwością zatrzymywania się na piętrach pośrednich w przypadku zgłoszenia takiego żądania przez pasażerów.

Projekt zakładał również obsługę sygnałów alarmowych oraz awaryjnych, co jest niezbędnym elementem ze względu na bezpieczeństwo pasażerów. Dodatkowo zaimplementowano funkcję wizualizacji stanu wyświetlacza windy poprzez zapis informacji do pliku tekstowego.

## 1.2 Założenia projektowe

W ramach projektu przyjęto kilka założeń technicznych, które determinowały sposób działania sterownika w różnych sytuacjach. Przede wszystkim ustalono, że system będzie obsługiwał dokładnie pięć pięter, co przekłada się na konieczność reprezentacji numeru piętra za pomocą trzech bitów w wektorze logicznym. Takie rozwiązanie pozwala na zakodowanie wartości od zera do czterech, które odpowiadają poszczególnym kondygnacjom budynku.

Czas przejazdu między sąsiednimi piętrami został ustalony na sześć taktów zegarowych. Wartość ta została dobrana w taki sposób, aby symulacja była wystarczająco szybka do celów testowych, a jednocześnie pozwalała na obserwację wszystkich stanów pośrednich podczas przejazdu. W rzeczywistym systemie windowym czas ten byłby znacznie dłuższy i zależny od parametrów fizycznych windy.

Czas otwarcia drzwi został ustalony na dziesięć taktów zegarowych, po których drzwi zamykają się automatycznie. Pasażer może zamknąć drzwi wcześniej poprzez naciśnięcie przycisku zamknięcia. Rozwiązanie to odzwierciedla rzeczywiste zachowanie wind, gdzie drzwi pozostają otwarte przez określony czas, umożliwiając wsiadanie i wysiadanie pasażerów, a następnie zamykają się samoczynnie lub na żądanie pasażera.

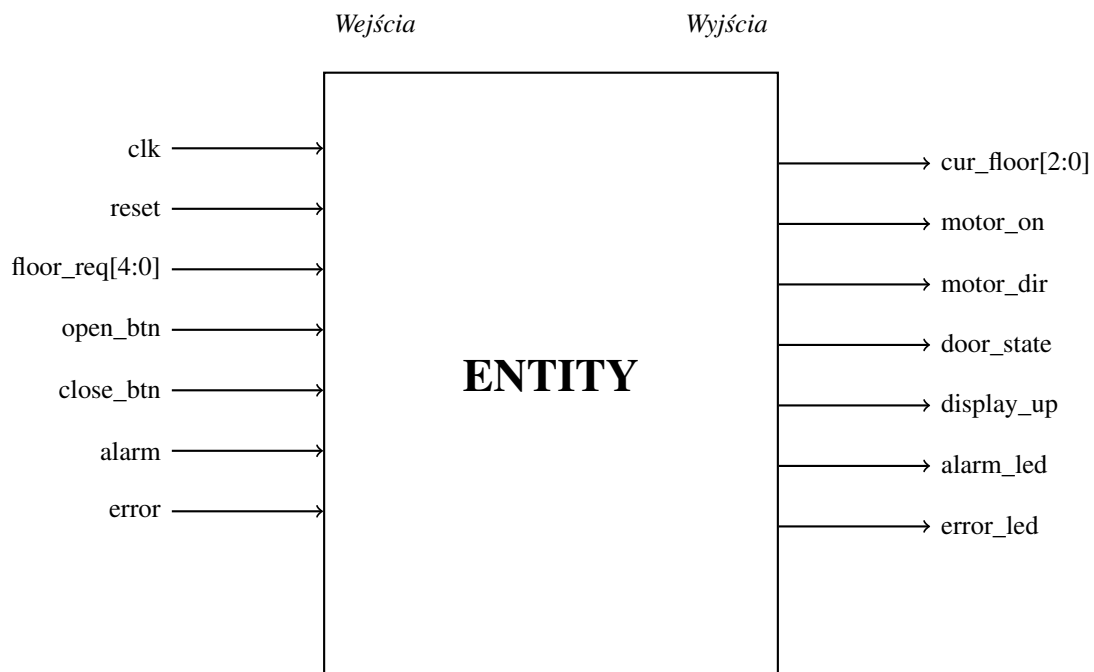
Obsługa żądań pięter została zaimplementowana z wykorzystaniem systemu priorytetów. Winda realizuje przede wszystkim żądania w aktualnym kierunku jazdy, zatrzymując się na piętrach pośrednich, jeśli były takie żądania. Dopiero po obsłużeniu wszystkich żądań w danym kierunku następuje zmiana kierunku jazdy i obsługa pozostałych żądań. Takie podejście minimalizuje liczbę zmian kierunku i optymalizuje czas przejazdu.

Obsługa dwóch rodzajów stanów awaryjnych. Pierwszym jest sygnał alarmu, który powoduje natychmiastowe zatrzymanie windy oraz otwarcie drzwi, umożliwiając pasażerom opuszczenie kabiny. Po ustąpieniu alarmu system automatycznie wraca do normalnego trybu pracy. Drugim rodzajem jest sygnał błędu mechanicznego, który powoduje całkowite zatrzymanie systemu. W tym przypadku wznowienie pracy wymaga usunięcia awarii oraz wykonania resetu systemu, co odpowiada procedurze stosowanej w rzeczywistych systemach windowych wymagających interwencji serwisowej.

W projekcie przyjęto dwa uproszczenia. Pierwsze polega na pominięciu sterowania drzwiami szybowymi i przyjęciu, że w momencie otwarcia drzwi windy, otwierają się odpowiednie drzwi szybowe. Drugie polega na przyjęciu, że wektor przycisków żądań pięter jest tym samym wektorem, co przycisków wezwań windy na poszczególnych piętrach.

## 2. Schemat jednostki projektowej

Jednostka projektowa sterownika windy została zaprojektowana z uwzględnieniem wszystkich niezbędnych sygnałów wejściowych i wyjściowych, umożliwiających pełną kontrolę nad pracą windy oraz monitorowanie jej stanu. Rysunek 1 przedstawia schemat blokowy jednostki projektowej wraz z opisem wszystkich portów.



Rysunek 1: Schemat blokowy entity sterownika windy

## 2.1 Opis portów wejściowych

W tabeli 1 przedstawiono szczegółowy opis wszystkich portów wejściowych wraz z ich typami oraz funkcjami.

Sygnał zegarowy `clk` stanowi podstawę synchronizacji całego układu. Wszystkie operacje sekwencyjne w sterowniku są wykonywane na zboczu narastającym tego sygnału, co zapewnia przewidywalne zachowanie systemu.

Sygnał `reset` jest asynchronicznym sygnałem resetującym, który przywraca układ do stanu początkowego. Po aktywacji tego sygnału winda zostaje ustawiona na parterze, wszystkie żądania są kasowane, a maszyna stanów przechodzi do stanu bezczynności. Asynchroniczny charakter resetu oznacza, że zadziała on natychmiast, niezależnie od stanu sygnału zegarowego.

Wektor `floor_req` o szerokości pięciu bitów reprezentuje żądania poszczególnych pięter. Każdy bit tego wektora odpowiada jednemu piętru, gdzie bit zerowy reprezentuje parter, a bit czwarty reprezentuje najwyższe piętro. Aktywacja danego bitu oznacza zgłoszenie żądania jazdy na odpowiednie piętro. Żądania są zatraskiwane w rejestrze wewnętrznym, co oznacza, że nawet krótkotrwałe naciśnięcie przycisku zostanie zapamiętane przez system.

Sygnały `open_btn` oraz `close_btn` reprezentują przyciski otwierania i zamykania drzwi znajdujące się w kabinie windy. Przycisk otwierania pozwala pasażerowi otworzyć drzwi, gdy winda stoi, oraz przedłużyć czas ich otwarcia. Przycisk zamykania umożliwia natychmiastowe zamknięcie drzwi bez czekania na automatyczne zamknięcie.

Sygnały `alarm` oraz `error` reprezentują odpowiednio sygnał alarmu oraz sygnał awarii mechanicznej. Są one podłączone do odpowiednich czujników i systemów bezpieczeństwa windy. Aktywacja któregośkolwiek z tych sygnałów powoduje natychmiastową reakcję sterownika, zgodną z przyjętymi procedurami bezpieczeństwa.

*Tabela 1: Opis portów wejściowych*

| Nazwa                  | Typ                                | Opis                                                                                   |
|------------------------|------------------------------------|----------------------------------------------------------------------------------------|
| <code>clk</code>       | <code>std_logic</code>             | Sygnał zegarowy systemu                                                                |
| <code>reset</code>     | <code>std_logic</code>             | Asynchroniczny sygnał resetujący układ do stanu początkowego                           |
| <code>floor_req</code> | <code>std_logic_vector(4:0)</code> | Żądania pięter – każdy bit odpowiada jednemu piętru (bit 0 = parter, bit 4 = piętro 4) |
| <code>open_btn</code>  | <code>std_logic</code>             | Przycisk otwierania drzwi                                                              |
| <code>close_btn</code> | <code>std_logic</code>             | Przycisk zamykania drzwi                                                               |
| <code>alarm</code>     | <code>std_logic</code>             | Sygnał alarmu                                                                          |
| <code>error</code>     | <code>std_logic</code>             | Sygnał awarii mechanicznej                                                             |

## 2.2 Opis portów wyjściowych

W tabeli 2 przedstawiono szczegółowy opis wszystkich portów wyjściowych.

Sygnał `cur_floor` jest 3-bitowym wektorem reprezentującym, na którym piętrze znajduje się winda. Wartość tego sygnału zmienia się, gdy kabina osiąga kolejne piętra.

Sygnały `motor_on` oraz `motor_dir` sterują pracą silnika napędowego windy. Sygnał `motor_on` określa, czy silnik jest włączony, natomiast `motor_dir` określa kierunek jazdy. Wartość logiczna jeden na sygnale `motor_dir` oznacza jazdę w górę, a wartość zero oznacza jazdę w dół.

Sygnał `door_state` informuje o stanie drzwi kabiny. Wartość logiczna jeden oznacza, że drzwi są otwarte, natomiast wartość zero oznacza, że drzwi są zamknięte.

Sygnał `display_up` służy do aktywacji wyświetlacza w kabinie windy. Gdy sygnał ten jest aktywny, wyświetlacz pokazuje informacje o aktualnym stanie windy, takie jak numer piętra czy kierunek jazdy.

Sygnały `alarm_led` oraz `error_led` sterują diodami, które sygnalizują informacje odpowiednio dla stanu alarmu i stanu awarii. Aktywacja tych sygnałów zapewnia wizualną informację dla pasażerów.

Tabela 2: Opis portów wyjściowych

| Nazwa                   | Typ                                | Opis                                        |
|-------------------------|------------------------------------|---------------------------------------------|
| <code>cur_floor</code>  | <code>std_logic_vector(2:0)</code> | Aktualne piętro windy (0-4)                 |
| <code>motor_on</code>   | <code>std_logic</code>             | Praca silnika (1 = włączony, 0 = wyłączony) |
| <code>motor_dir</code>  | <code>std_logic</code>             | Kierunek jazdy (1 = góra, 0 = dół)          |
| <code>door_state</code> | <code>std_logic</code>             | Stan drzwi (1 = otwarte, 0 = zamknięte)     |
| <code>display_up</code> | <code>std_logic</code>             | Aktywacja wyświetlacza                      |
| <code>alarm_led</code>  | <code>std_logic</code>             | Dioda sygnalizująca stan alarmu             |
| <code>error_led</code>  | <code>std_logic</code>             | Dioda sygnalizująca stan awarii             |

## 3. Algorytm pracy układu

Na rysunku 2 przedstawiono diagram maszyny stanów sterownika windy. Diagram ten ilustruje wszystkie możliwe stany systemu oraz przejścia między nimi, wraz z warunkami, które muszą być spełnione, aby dane przejście nastąpiło. Analiza diagramu pozwala na zrozumienie logiki działania sterownika oraz przewidzenie jego zachowania w różnych sytuacjach. Z każdego stanu można przejść do stanu `ERROR` i `ALARM`, lecz stan `ERROR` ma priorytet.

Maszyna stanów sterownika windy składa się z dziewięciu stanów, z których każdy odpowiada za realizację określonej funkcji systemu.

**Stan IDLE (bezczynność)** jest stanem początkowym oraz stanem spoczynkowym

maszyny stanów. Winda przechodzi do tego stanu po zakończeniu obsługi wszystkich żądań lub bezpośrednio po resecie systemu. W stanie tym silnik napędowy jest wyłączony, drzwi są zamknięte, a wyświetlacz jest nieaktywny. Stan IDLE pełni funkcję oczekiwania na nowe zdarzenia. System pozostaje w nim do momentu pojawienia się żądania piętra, naciśnięcia przycisku otwarcia drzwi lub wystąpienia sytuacji awaryjnej.

**Stan DECIDE (decyzja)** jest stanem przejściowym odpowiedzialnym za podjęcie decyzji o kierunku jazdy windy. System przechodzi do tego stanu ze stanu IDLE, gdy zostanie wykryte żądanie piętra w rejestrze żądań. W stanie DECIDE następuje analiza wszystkich zgłoszonych żądań względem aktualnej pozycji windy. Algorytm sprawdza najpierw, czy istnieją żądania pięter położonych powyżej aktualnej pozycji, a następnie czy istnieją żądania pięter położonych poniżej. Na podstawie tej analizy podejmowana jest decyzja o przejściu do stanu GO\_UP (jazda w górę), GO\_DOWN (jazda w dół) lub OPEN\_DOOR (jeśli żądanie dotyczy aktualnego piętra). Stan ten jest niezbędny do prawidłowej priorytetyzacji żądań i optymalizacji trasy windy.

**Stan GO\_UP (jazda w górę)** realizuje fizyczny ruch kabiny windy w kierunku wyższych pięter. Po wejściu do tego stanu następuje włączenie silnika napędowego oraz ustawienie kierunku jazdy na wartość odpowiadającą ruchowi w górę. W trakcie przebywania w tym stanie działa wewnętrzny licznik move\_cnt, który odlicza takty zegarowe potrzebne do osiągnięcia kolejnego piętra. Po odliczeniu sześciu taktów licznik jest zerowany, rejestr aktualnego piętra jest inkrementowany, a system sprawdza, czy na nowo osiągniętym piętrze istnieje żądanie zatrzymania. Jeśli tak, następuje przejście do stanu OPEN\_DOOR. W przeciwnym przypadku winda kontynuuje jazdę. Stan GO\_UP może być również opuszczony w przypadku wystąpienia alarmu lub błędu, co zapewnia natychmiastową reakcję na sytuacje awaryjne podczas jazdy.

**Stan GO\_DOWN (jazda w dół)** działa analogicznie do stanu GO\_UP, z tą różnicą, że realizuje ruch kabiny w kierunku niższych pięter. Silnik jest włączony, a kierunek jazdy ustawiony na wartość odpowiadającą ruchowi w dół. Licznik move\_cnt odlicza takty między piętrami, po czym rejestr aktualnego piętra jest dekrementowany. Podobnie jak w przypadku jazdy w górę, system sprawdza na każdym osiągniętym piętrze, czy istnieje żądanie zatrzymania, i w razie potrzeby przechodzi do stanu OPEN\_DOOR.

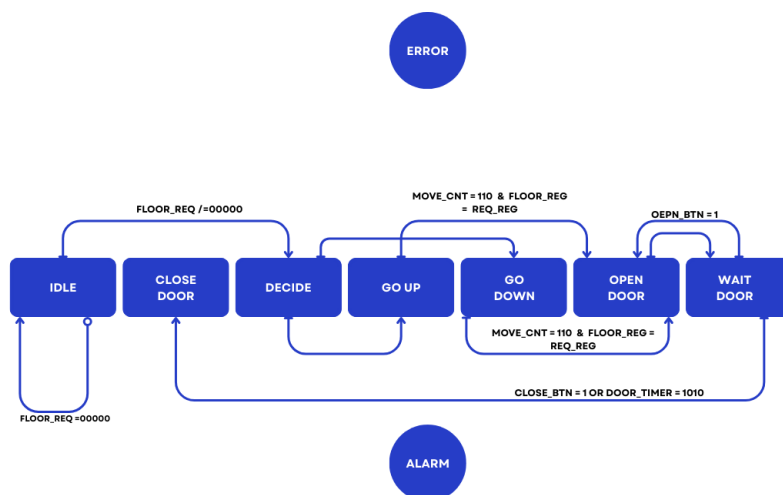
**Stan OPEN\_DOOR (otwieranie drzwi)** jest krótkotrwałym stanem przejściowym, którego zadaniem jest zainicjowanie procesu otwierania drzwi kabiny. Po wejściu do tego stanu sygnał door\_state przyjmuje wartość logiczną jeden, co oznacza otwarcie drzwi. Jednocześnie następuje kasowanie żądania dla aktualnego piętra w rejestrze żądań. Stan OPEN\_DOOR trwa tylko jeden takt zegarowy, po czym system przechodzi automatycznie do stanu WAIT\_DOOR. **Stan WAIT\_DOOR (oczekiwanie z otwartymi drzwiami)** reprezentuje okres, w którym drzwi kabiny są otwarte i pasażerowie mogą wsiadać lub wysiadać. W tym stanie działa wewnętrzny licznik door\_timer, który odlicza czas do automatycznego zamknięcia drzwi. Domyślny czas oczekiwania wynosi dziesięć taktów zegarowych. Pasażer może jednak skrócić ten czas poprzez naciśnięcie przycisku zamknięcia drzwi (close\_btn), co

powoduje natychmiastowe przejście do stanu CLOSE\_DOOR. Alternatywnie, naciśnięcie przycisku otwarcia drzwi (open\_btn) powoduje powrót do stanu OPEN\_DOOR, co resetuje licznik i przedłuża czas otwarcia drzwi.

**Stan CLOSE\_DOOR (zamykanie drzwi)** jest kolejnym stanem przejściowym, który finalizuje proces obsługi piętra. Po wejściu do tego stanu drzwi zostają zamknięte (door\_state przyjmuje wartość zero). Następnie system sprawdza, czy w rejestrze żądań pozostały jakiegokolwiek nieobsłużone żądania. Jeśli tak, następuje przejście do stanu DECIDE w celu podjęcia decyzji o dalszym kierunku jazdy. Jeśli wszystkie żądania zostały obsłużone, system przechodzi do stanu IDLE i oczekuje na nowe zdarzenia.

**Stan ALARM\_S (alarm)** jest specjalnym stanem awaryjnym, do którego system przechodzi po wykryciu aktywnego sygnału alarmu. Przejście do tego stanu może nastąpić z każdego innego stanu, z wyjątkiem ERROR\_S, co zapewnia szybką reakcję na sytuacje zagrożenia. Po wejściu do stanu ALARM\_S następuje natychmiastowe wyłączenie silnika napędowego oraz otwarcie drzwi kabiny, umożliwiając pasażerom ewakuację. Jednocześnie aktywowana jest dioda sygnalizacyjna alarmu (alarm\_led). System pozostaje w tym stanie do momentu dezaktywacji sygnału alarmu, po czym automatycznie powraca do stanu IDLE. Nie jest wymagany reset systemu, co pozwala na szybkie wznowienie normalnej pracy po ustąpieniu zagrożenia.

**Stan ERROR\_S (awaria mechaniczna)** jest stanem awaryjnym, zarezerwowanym dla poważnych usterek mechanicznych windy. Przejście do tego stanu następuje po wykryciu aktywnego sygnału błędu i może nastąpić z każdego innego stanu, włącznie ze stanem ALARM\_S. Po wejściu do stanu ERROR\_S silnik napędowy jest natychmiast wyłączany, a dioda sygnalizacyjna awarii (error\_led) zostaje aktywowana. W odróżnieniu od stanu alarmu, w stanie ERROR\_S drzwi pozostają w swoim aktualnym położeniu i nie są automatycznie otwierane. System pozostaje zablokowany w tym stanie do momentu jednoczesnego usunięcia awarii (error = '0') oraz wykonania resetu (reset = '1').



Rysunek 2: Graf maszyny stanów sterownika windy

W tabeli 3 przedstawiono zestawienie wszystkich stanów maszyny stanów w formie skróconej dla szybkiego odniesienia.

Tabela 3: Zestawienie stanów maszyny stanów

| Stan       | Opis                                                                                              |
|------------|---------------------------------------------------------------------------------------------------|
| IDLE       | Stan bezczynności. Winda oczekuje na żądania. Silnik wyłączony, drzwi zamknięte.                  |
| DECIDE     | Wybór przez sterownik kierunku jazdy. Aktualizacja rejestru piętra docelowego na podstawie żądań. |
| GO_UP      | Jazda w górę. Silnik włączony, motor_dir = '1'. Licznik taktów między piętrami (6 taktów).        |
| GO_DOWN    | Jazda w dół. Silnik włączony, motor_dir = '0'. Licznik taktów między piętrami (6 taktów).         |
| OPEN_DOOR  | Otwieranie drzwi. Kasowanie żądania dla aktualnego piętra.                                        |
| WAIT_DOOR  | Oczekiwanie z otwartymi drzwiami. Licznik czasu do automatycznego zamknięcia (10 taktów).         |
| CLOSE_DOOR | Zamykanie drzwi. Sprawdzenie kolejnych żądań.                                                     |
| ALARM_S    | Stan alarmu. Zatrzymanie silnika, otwarcie drzwi, sygnalizacja LED.                               |
| ERROR_S    | Stan awarii. Całkowite zatrzymanie systemu, sygnalizacja LED.                                     |



### 3.1 Opis procesów

Architektura sterownika została zrealizowana z wykorzystaniem trzech odrębnych procesów, z których każdy odpowiada za określony aspekt działania systemu.

Pierwszy proces, nazwany procesem wyświetlacza, jest odpowiedzialny za wizualizację stanu windy poprzez zapis informacji do pliku tekstowego. Proces ten działa synchronicznie z sygnałem zegarowym i przy każdym zboczu narastającym zapisuje do pliku aktualny numer piętra oraz informację o stanie systemu. W zależności od aktualnego stanu maszyny stanów, proces wyświetla odpowiedni symbol: strzałkę w górę dla stanu GO\_UP, strzałkę w dół dla stanu GO\_DOWN, napis "ALARM" dla stanu ALARM\_S lub napis "AWARIA" dla stanu ERROR\_S. W pozostałych stanach wyświetlany jest tylko numer piętra.

Drugi proces, nazywany procesem rejestrów, realizuje wszystkie operacje na rejestrach wewnętrznych. Jest to proces wrażliwy na zbocze narastające sygnału zegarowego oraz na asynchroniczny sygnał resetu. W przypadku aktywacji resetu następuje inicjalizacja wszystkich rejestrów do wartości początkowych: stan maszyny ustawiany jest na IDLE, rejestr piętra na zero (parter), rejestry żądań i piętra docelowego są zerowane, a wszystkie liczniki przyjmują wartość zero. Przy normalnej pracy, na każdym zboczu narastającym zegara proces wykonuje szereg operacji. Przede wszystkim aktualizuje stan maszyny stanów na podstawie wartości sygnału next\_stan obliczonego przez proces kombinacyjny. Następnie realizuje zatraskiwanie żądań pięter poprzez operację logiczną OR między aktualnym rejestrem żądań a wejściem floor\_req. W stanie OPEN\_DOOR proces kasuje żądanie odpowiadające aktualnemu piętru. W stanach GO\_UP i GO\_DOWN obsługuje licznik ruchu move\_cnt, inkrementując go przy każdym takcie. Po zakończeniu odliczania następuje aktualizacja rejestru piętra przez inkrementację lub dekrementację, w zależności od kierunku jazdy. W stanie WAIT\_DOOR proces obsługuje licznik czasu otwarcia drzwi door\_timer, inkrementując go przy każdym takcie.

Trzeci proces jest procesem kombinacyjnym, odpowiedzialnym za logikę przejść między stanami oraz za sterowanie wyjściami układu. Proces ten jest wrażliwy na zmiany wszystkich sygnałów wejściowych oraz sygnałów wewnętrznych, od których zależy logika przejść. Na początku procesu wszystkie wyjścia oraz sygnał next\_stan przyjmują wartości domyślne: silnik wyłączony, kierunek w dół, drzwi zamknięte, wyświetlacz włączony, diody sygnalizacyjne wyłączone, a następny stan równy aktualnemu. Następnie, w zależności od aktualnego stanu, proces określa odpowiednie wartości wyjść oraz warunki przejść do innych stanów. Szczególną uwagę zwrócono na priorytetyzację sygnałów awaryjnych: w każdym stanie najpierw sprawdzane są sygnały error i alarm, co zapewnia natychmiastową reakcję na sytuacje awaryjne, niezależnie od bieżącej operacji wykonywanej przez windę.

#### 4. Tabela sygnałów i zmiennych

W projekcie sterownika windy wykorzystano szereg sygnałów wewnętrznych, które przechowują informacje o stanie systemu oraz pełnią funkcje pomocnicze w realizacji logiki sterowania. Sygnały te są niezbędne do prawidłowego funkcjonowania maszyny stanów oraz do komunikacji między procesami w architekturze układu. W tabeli 4 przedstawiono szczegółowe zestawienie wszystkich sygnałów wewnętrznych wraz z ich typami, zakresami wartości oraz przeznaczeniem.

Sygnały stan oraz next\_stan przechowują odpowiednio aktualny i następny stan maszyny stanów.

Sygnał floor\_reg jest trzbitowym wektorem przechowującym numer aktualnego piętra windy. Zakres wartości od zera do czterech odpowiada pięciu obsługiwanym piętrům. Sygnał ten jest aktualizowany podczas jazdy windy, gdy kabina osiąga kolejne piętra.

Sygnał req\_reg jest pięciobitowym wektorem służącym jako rejestr zatrzaśniętych żądań pięter. Każdy bit odpowiada jednemu piętru, gdzie bit zero reprezentuje parter, a bit cztery reprezentuje najwyższe piętro. Wartość logiczna jeden na danej pozycji oznacza aktywne żądanie, które zostanie skasowane dopiero po otwarciu drzwi na odpowiednim piętrze.

Sygnał target\_floor przechowuje numer piętra docelowego, który jest wyznaczany w stanie DECIDE na podstawie analizy rejestru żądań. Wartość tego sygnału jest wykorzystywana do określenia momentu, w którym winda powinna zatrzymać się i otworzyć drzwi.

Sygnał move\_cnt jest 3-bitowym licznikiem odliczającym takty zegarowe podczas jazdy windy między piętrami.

Sygnał door\_timer jest czterobitowym licznikiem odmierzającym czas otwarcia drzwi.

Tabela 4: Zestawienie sygnałów wewnętrznych

| Nazwa        | Typ                   | Zakres      | Zastosowanie                        |
|--------------|-----------------------|-------------|-------------------------------------|
| stan         | STANY                 | 9 stanów    | Aktualny stan maszyny stanów        |
| next_stan    | STANY                 | 9 stanów    | Następny stan maszyny stanów        |
| floor_reg    | std_logic_vector(2:0) | 0–4         | Rejestr aktualnego piętra           |
| req_reg      | std_logic_vector(4:0) | 00000–11111 | Rejestr zatrzaśniętych żądań pięter |
| target_floor | std_logic_vector(2:0) | 0–4         | Rejestr piętra docelowego           |
| move_cnt     | std_logic_vector(2:0) | 0–5         | Licznik taktów między piętrami      |
| door_timer   | std_logic_vector(3:0) | 0–10        | Licznik czasu otwarcia drzwi        |

W tabeli 5 przedstawiono definicję typu STANY wraz z listą wszystkich wartości.

Tabela 5: Definicja typu STANY

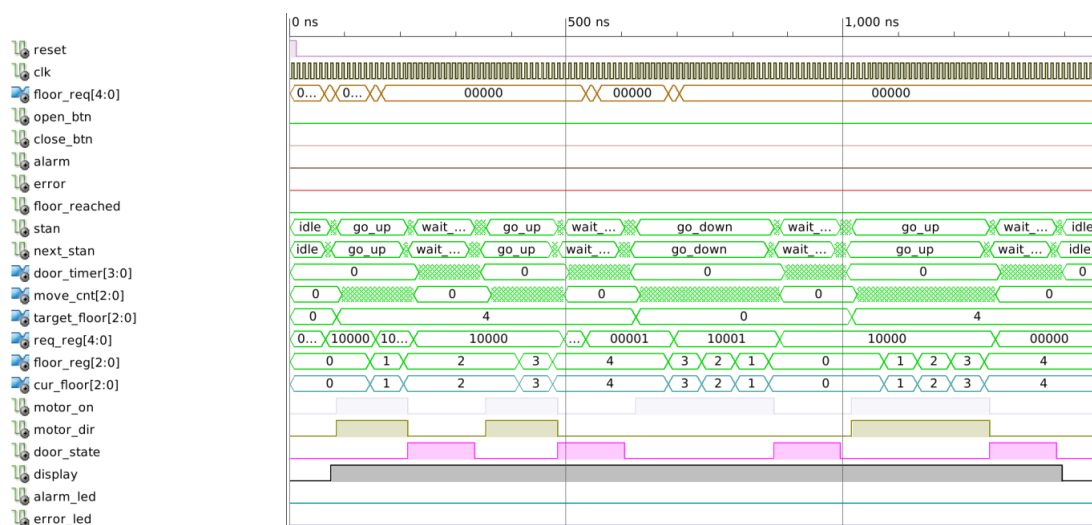
| Typ   | Wartości                                                                         |
|-------|----------------------------------------------------------------------------------|
| STANY | IDLE, DECIDE, GO_UP, GO_DOWN, OPEN_DOOR, WAIT_DOOR, CLOSE_DOOR, ALARM_S, ERROR_S |

## 5. Wyniki symulacji

W ramach weryfikacji poprawności działania zaprojektowanego sterownika windy przeprowadzono kilka testów symulacyjnych. Testy zostały zaprojektowane w taki sposób, aby sprawdzić wszystkie kluczowe aspekty działania systemu, włącznie z normalną pracą, obsługą wielu żądań jednocześnie, reakcją na sytuacje awaryjne oraz obsługą przycisków drzwi.

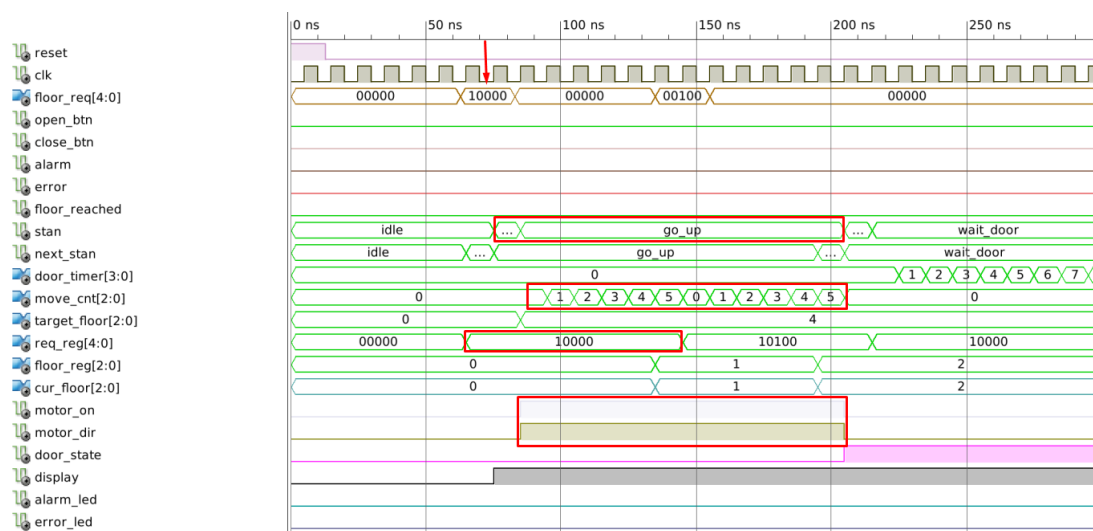
### 5.1 Scenariusz 1: Normalny przejazd góra-dół z zatrzymaniem na piętrze pośrednim

Pierwszy scenariusz testowy miał na celu weryfikację podstawowej funkcjonalności windy, obejmującej jazdę w górę z zatrzymaniem na piętrze pośrednim oraz powrót na parter. Scenariusz ten pozwolił sprawdzić poprawność działania mechanizmu priorytetyzacji żądań, zatrzymywania na piętrach pośrednich oraz zmiany kierunku jazdy. Na rysunku 3 przedstawiono przebieg czasowy symulacji dla tego scenariusza.



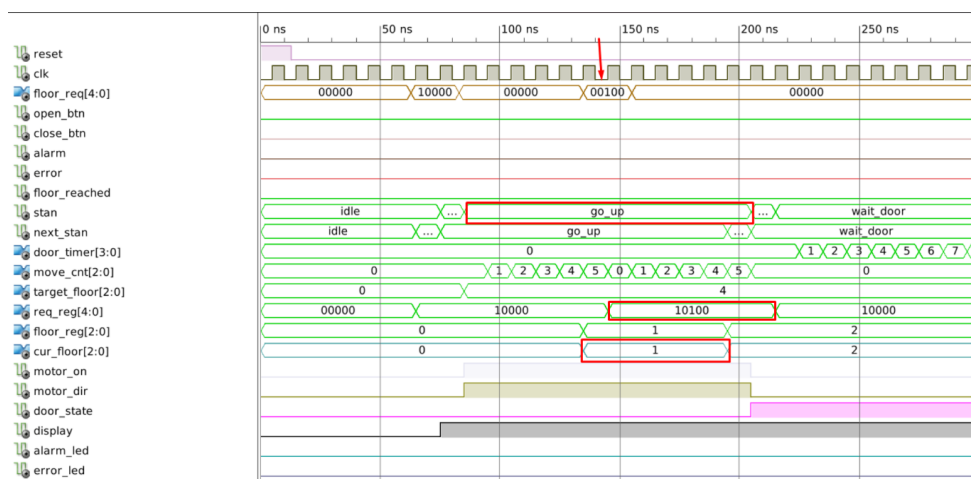
Rysunek 3: Przebieg czasowy symulacji - normalny przejazd góra-dół

Przedstawiony przebieg na rysunku 3 został podzielony, aby lepiej przedstawić symulację. Na rysunku 4 Test rozpoczyna się od aktywacji sygnału reset, który inicjalizuje system do stanu początkowego. Po resecie winda znajduje się na parterze (floor\_reg = "000") w stanie IDLE, z wyłączonym silnikiem i zamkniętymi drzwiami. W zaznaczonym strzałką momencie, pojawia się żądanie piętra poprzez ustawienie floor\_req na wartość "10000". System natychmiast wykrywa to żądanie, zatrzymuje je w rejestrze req\_reg i przechodzi do stanu DECIDE, który zdecydował o poruszaniu się w górę i przeszedł do stanu go\_up. W tym samym momencie został uruchomiony silnik oraz odmierzenie czasu przejazdu między piętrami.



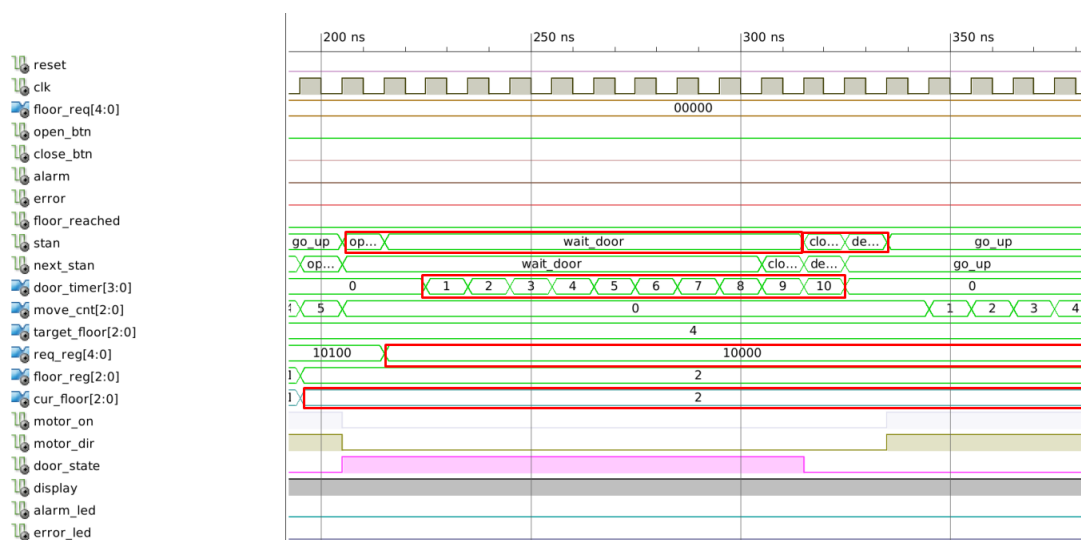
Rysunek 4: Przebieg czasowy symulacji - Decyzja o jeździe w górę

W czasie jazdy pojawiło się żądanie piętra drugiego, co przedstawiono na rysunku 5. Żądanie to zostało dodane do rejestru req\_reg.



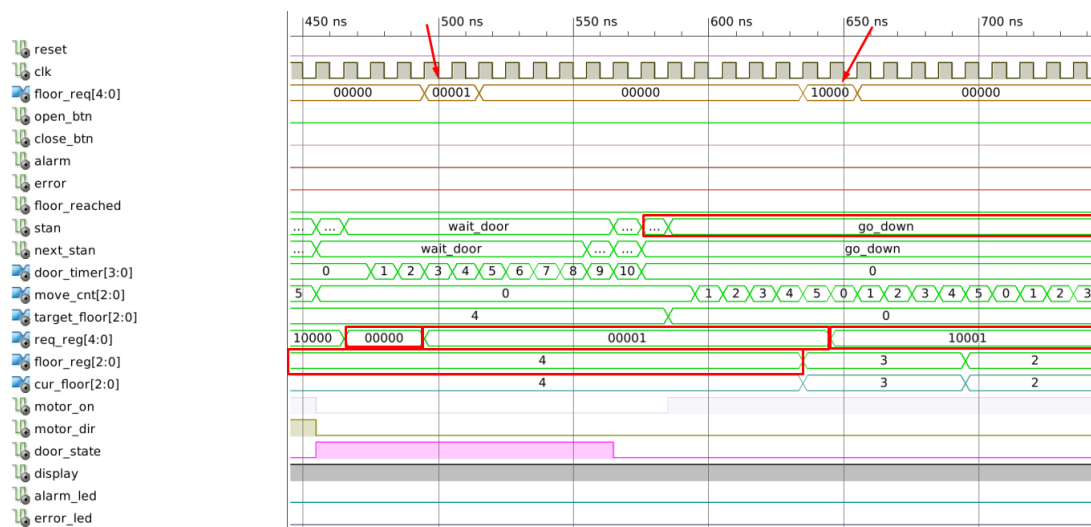
Rysunek 5: Przebieg czasowy symulacji - pojawienie się piętra pośredniego

Po dotarciu na drugie piętro (rysunek 6) sterownik usunął żądanie drugiego piętra z req\_reg, a drzwi windy zostały otworzone. Po otwarciu drzwi winda przeszła do stanu wait\_door i po 10 taktach door\_timer samoczynnie zamknęła drzwi, a następnie kontynuowała jazdę na czwarte piętro.



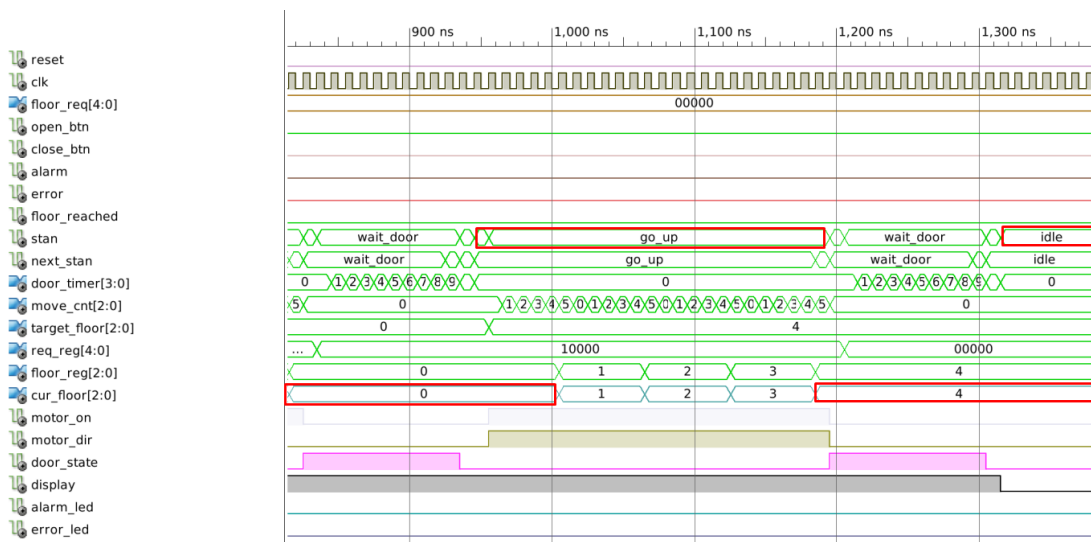
Rysunek 6: Przebieg czasowy symulacji - otwarcie drzwi

Po dotarciu na czwarte piętro procedura z otwieraniem drzwi została powtórzona. Na rysunku 7 możemy zauważyć, że po dotarciu windy wyczyszczony został rejestr req\_reg. W międzyczasie, gdy drzwi windy były otwarte, winda została wezwana na parter i żądanie zostało zapisane do rejestru req\_reg. Po zamknięciu drzwi winda przeszła do stanu decide, gdzie podjęto decyzję o jeździe w dół, uruchomieniu silnika oraz jego kierunku w dół. Gdy winda kierowała się w dół, pojawiło się żądanie, aby wróciła na czwarte piętro, co również zostało dopisane do rejestru żądań.



Rysunek 7: Przebieg czasowy symulacji - powrót na parter

Winda poprawnie zignorowała drugie żądanie i kierowała się aż do parteru (rysunek 8), gdzie ponownie otworzyła drzwi, usunęła obsłużone żądanie z rejestru req\_reg, a następnie wyruszyła do drugiego żądania. Po dotarciu na piąte i otwarciu drzwi wszystkie żądania zostają obsłużone, a system przechodzi do stanu IDLE, oczekując na kolejne żądania.



Rysunek 8: Przebieg czasowy symulacji - koniec przejazdu

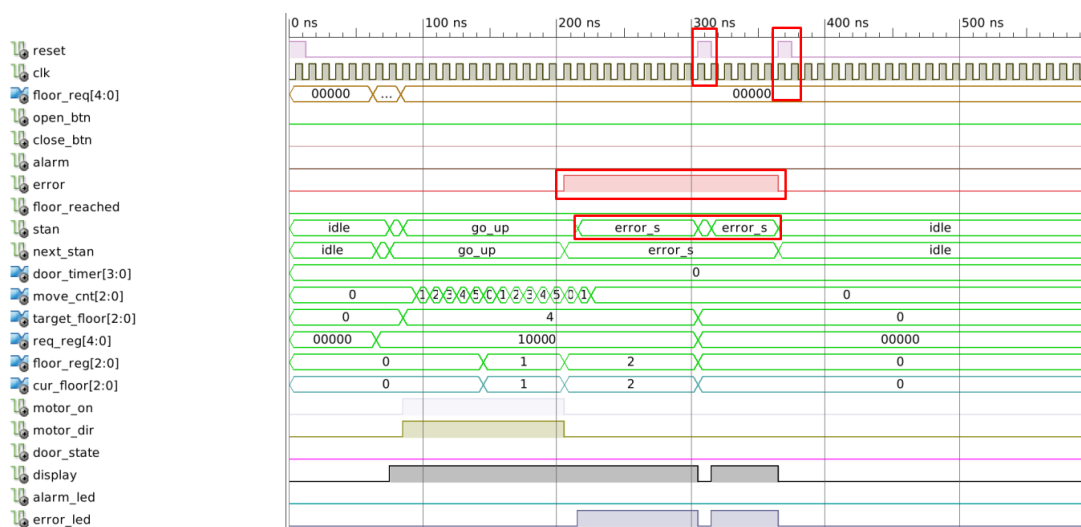
## 5.2 Scenariusz 2: Awaria mechaniczna podczas jazdy

Drugi scenariusz testowy miał na celu weryfikację reakcji systemu na sygnał awarii mechanicznej występujący podczas jazdy windy. Na rysunku 9 przedstawiono przebieg czasowy symulacji dla tego scenariusza.

Test rozpoczyna się od zgłoszenia żądania piętra czwartego, na co winda reaguje rozpoczęciem jazdy w górę. W trakcie jazdy, gdy kabina znajduje się na wysokości drugiego piętra, zostaje aktywowany sygnał błędu mechanicznego ( $error = '1'$ ), symulujący wykrycie awarii przez czujnik systemu bezpieczeństwa.

System natychmiast reaguje na sygnał błędu, przerywając jazdę w górę i przechodząc do stanu  $ERROR\_S$ . Przejście to następuje z najwyższym priorytetem, niezależnie od aktualnie wykonywanej operacji. Po wejściu do stanu awarii silnik napędowy zostaje natychmiast wyłączony ( $motor\_on = '0'$ ), co zatrzymuje ruch kabiny. Jednocześnie zostaje aktywowana dioda sygnalizacyjna awarii ( $error\_led = '1'$ ), informując o wystąpieniu problemu.

Próbowano wykonać reset, aby zlikwidować usterkę lecz bez usunięcia przyczyny awarii, winda ciągle wracała do stanu error. Dopiero po resecie i usunięciu awarii winda przeszła do stanu  $IDLE$ .



Rysunek 9: Przebieg czasowy symulacji - awaria mechaniczna

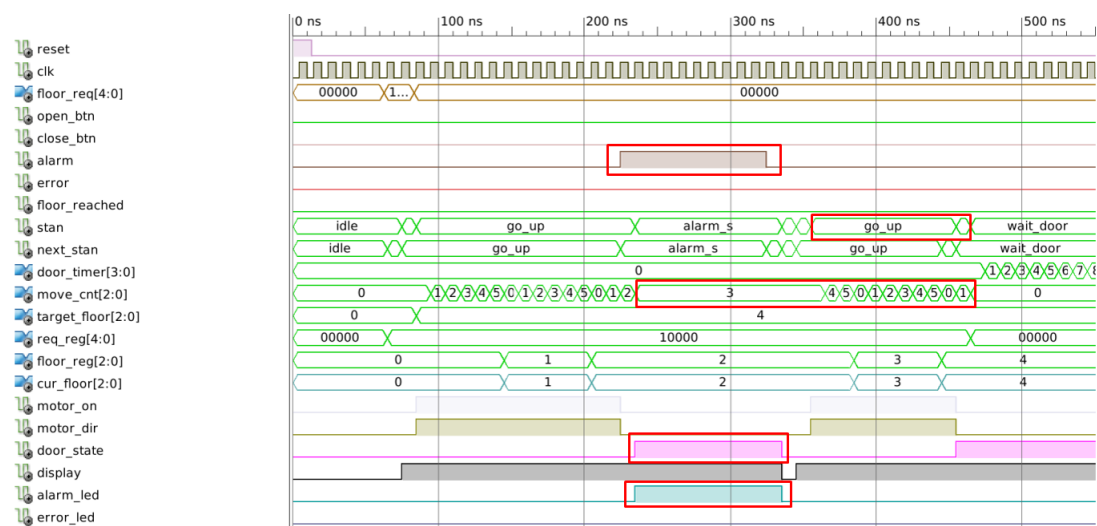
### 5.3 Scenariusz 3: Alarm podczas jazdy

Trzeci scenariusz testowy weryfikował reakcję systemu na sygnał alarmu występujący podczas jazdy windy. Sygnał alarmu reprezentuje sytuację awaryjną wymagającą szybkiej reakcji, ale mniej krytyczną niż awaria mechaniczna, na przykład naciśnięcie przycisku alarmowego przez pasażera. Na rysunku 10 przedstawiono przebieg czasowy symulacji.

Test przebiega podobnie do poprzedniego scenariusza w początkowej fazie. Winda rozpoczyna jazdę na piętro czwarte, a w trakcie jazdy, na wysokości drugiego piętra, zostaje aktywowany sygnał alarmu ( $alarm = '1'$ ).

System natychmiast reaguje przejściem do stanu ALARM\_S. Po wejściu do tego stanu następuje wyłączenie silnika napędowego ( $\text{motor\_on} = '0'$ ), zatrzymujące ruch kabiny. Kluczową różnicą w porównaniu ze stanem awarii jest automatyczne otwarcie drzwi ( $\text{door\_state} = '1'$ ), które umożliwia pasażerom opuszczenie kabiny w sytuacji zagrożenia. Jednocześnie aktywowana jest dioda sygnalizacyjna alarmu ( $\text{alarm\_led} = '1'$ ).

System pozostaje w stanie ALARM\_S przez cały okres aktywności sygnału alarmu. Po jego dezaktywacji ( $\text{alarm} = '0'$ ) system automatycznie przechodzi do kontynuowania jazdy bez konieczności wykonywania resetu.

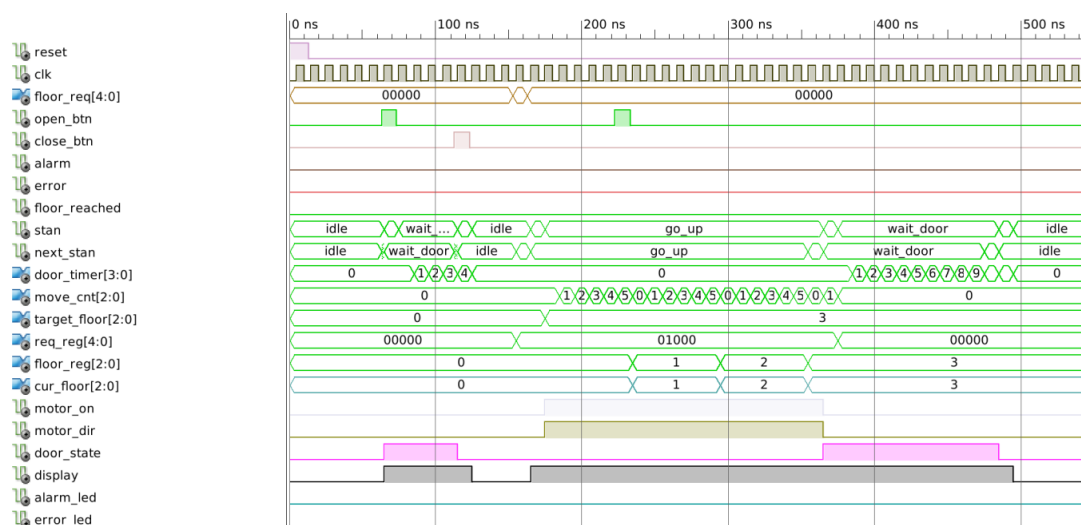


Rysunek 10: Przebieg czasowy symulacji – alarm podczas jazdy



## 5.4 Scenariusz 4: Obsługa przycisków otwierania i zamykania drzwi

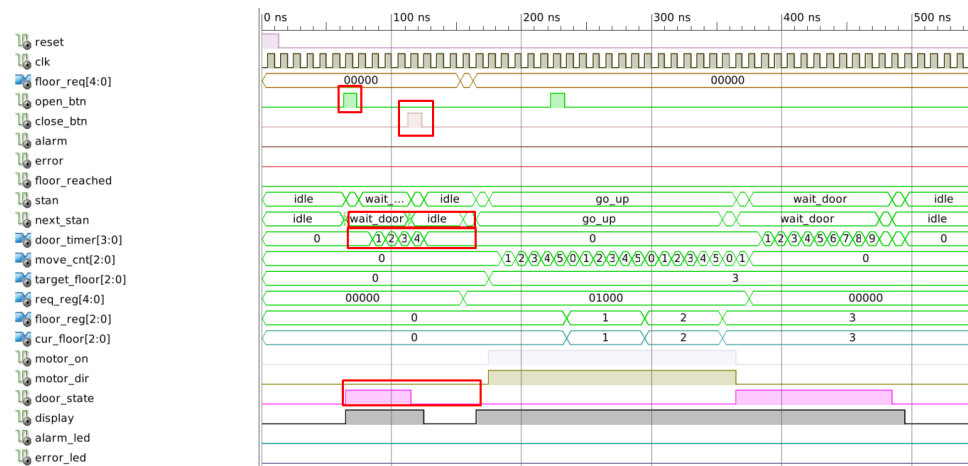
Czwarty scenariusz testowy koncentrował się na weryfikacji poprawności obsługi przycisków otwierania i zamykania drzwi w różnych sytuacjach. Testy obejmowały zarówno prawidłowe przypadki użycia tych przycisków, jak i próby ich użycia w sytuacjach, gdy powinny być ignorowane. Na rysunku 11 przedstawiono przebieg czasowy symulacji.



Rysunek 11: Przebieg czasowy symulacji – obsługa przycisków otwierania i zamykania drzwi

Test rozpoczyna się od sytuacji, w której winda stoi na parterze w stanie IDLE. Pasażer znajdujący się wewnątrz kabiny naciska przycisk otwarcia drzwi (`open_btn = '1'`). System poprawnie reaguje, przechodząc do stanu OPEN\_DOOR, a następnie WAIT\_DOOR, i otwierając drzwi kabiny (rysunek 12).

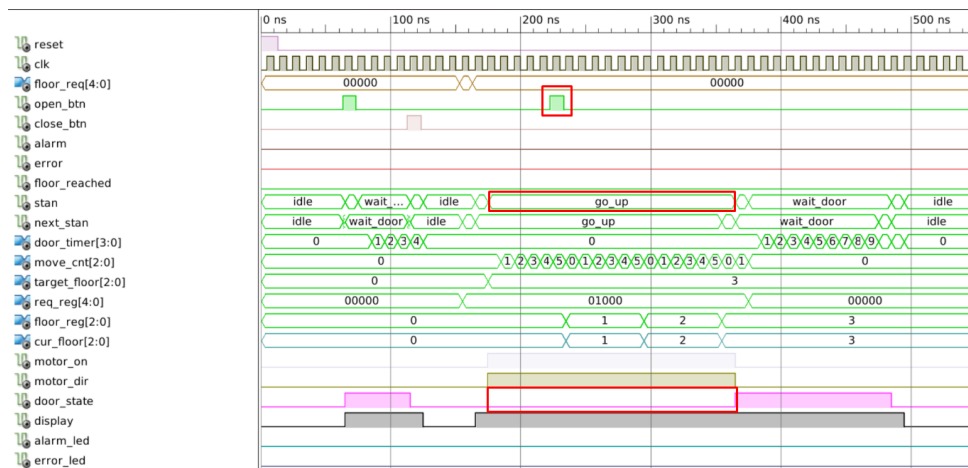
Nowy pasażer wybiera piętro docelowe za pomocą przycisku piętra, a następnie naciska przycisk zamknięcia drzwi (`close_btn = '1'`), chcąc przyspieszyć odjazd windy. System poprawnie reaguje na to żądanie, licznik `door_timer` przestaje odliczać, a drzwi zamykają się natychmiast, bez oczekiwania na automatyczne zamknięcie. Winda rozpoczyna jazdę do góry.



Rysunek 12: Przebieg czasowy symulacji – obsługa przycisków otwierania i zamykania drzwi

Kolejna faza testu weryfikuje zachowanie systemu, gdy przycisk otwarcia drzwi zostanie naciśnięty podczas jazdy windy (rysunek 13). Pasażer naciska open\_btn, jednak system poprawnie ignoruje to żądanie, ponieważ otwieranie drzwi podczas ruchu kabiny jest niedozwolone ze względów bezpieczeństwa. Winda kontynuuje jazdę bez żadnej reakcji na naciśnięcie przycisku.

Po dotarciu na piętro docelowe drzwi otwierają się automatycznie zgodnie z normalną procedurą. Test potwierdził, że mechanizm obsługi przycisków drzwi działa poprawnie, umożliwiając manualne sterowanie drzwiami, gdy jest to bezpieczne, i blokując niebezpieczne operacje podczas jazdy.



Rysunek 13: Przebieg czasowy symulacji – obsługa przycisków otwierania i zamykania drzwi

## **6. Dyskusja wyników i wnioski**

### **6.1 Wnioski**

Przeprowadzone symulacje potwierdziły poprawność działania zaprojektowanego sterownika windy we wszystkich testowanych scenariuszach. Wyniki testów wykazały, że system spełnia wszystkie założenia projektowe i prawidłowo realizuje wymagane funkcjonalności.

W zakresie obsługi żądań pięter winda poprawnie reaguje na zgłoszenia, realizując jazdę w górę i w dół z zachowaniem wymaganej liczby taktów zegarowych między piętrami. System prawidłowo priorytetyzuje żądania, kontynuując jazdę w aktualnym kierunku dopóki istnieją żądania w tym kierunku, a następnie zmieniając kierunek w celu obsługi pozostałych żądań. Mechanizm zatrzymywania na piętrach pośrednich działa prawidłowo, wykrywając żądania dla kolejnych pięter na trasie i otwierając drzwi w celu obsługi pasażerów.

## 7. Listing kodu źródłowego

Poniżej przedstawiono pełny kod źródłowy modułu sterownika windy.

```
1  --Autor: Jakub Szymanek 277673
2  --Sterownik windy w 5-pietrowym budynku
3  --Pt. 15:45
4  --26.01.2026r.
5  library IEEE;
6  use IEEE.STD_LOGIC_1164.ALL;
7  use IEEE.STD_LOGIC_UNSIGNED.ALL;
8  use std.textio.all;
9  use IEEE.std_logic_textio.all;
10
11 entity Winda is
12     port (
13         -- Wejscia
14         clk, reset : in std_logic; -- reset i zegar
15         floor_req  : in std_logic_vector(4 downto 0); -- Przyciski (0-4)
16         open_btn   : in std_logic; -- przycisk otwierania drzwi
17         close_btn  : in std_logic; -- przycisk zamykania drzwi
18         alarm      : in std_logic; -- wejscie alarmu
19         error      : in std_logic; -- wejscie od czujnika; blad
20                     mechaniczny windy
21         -- Wyjscia
22         cur_floor  : out std_logic_vector(2 downto 0); -- aktualne
23                     pietro
24         motor_on   : out std_logic; -- silnik 1=włączony 0=wylłączony
25         motor_dir  : out std_logic; -- 1 = gora, 0 = dol
26         door_state : out std_logic; -- drzwi 1 = otwarte 0 = zamkniete
27         display    : out std_logic; -- wyswietlanie na wyswietlaczu 1 =
28                     wlaczone , 0 = wylaczone
29         alarm_led  : out std_logic; -- zasilanie led od alarmu
30         error_led  : out std_logic; -- zasilanie led od awarii
31     );
32 end Winda;
33
34 architecture Behavioral of Winda is
35
36     type STANY is (IDLE, DECIDE, GO_UP, GO_DOWN, OPEN_DOOR, WAIT_DOOR,
37                     CLOSE_DOOR, ALARM_S, ERROR_S);
38     signal stan, next_stan : STANY;
39
40     -- Sygnały wewnętrzne
41     signal floor_reg : std_logic_vector(2 downto 0) := (others => '0');
42         -- Rejestr aktualnego pietra
43     signal req_reg : std_logic_vector(4 downto 0) := (others => '0');
```

```

-- Rejestr zatrzasnietych pieter
39 signal target_floor : std_logic_vector(2 downto 0) := (others => '0');
-- Rejestr pietro docelowe
40 signal move_cnt      : std_logic_vector(2 downto 0) := (others => '0');
-- licznik czasu jazdy windy
41 signal door_timer    : std_logic_vector(3 downto 0) := (others => '0');
-- licznik czasu otwarcia drzwi
42
43 begin
44
45     cur_floor <= floor_reg; -- przypisanie aktualnego pietra do wyjscia
46
47
48     wyswietlacz: process(clk)
49         file display : text open write_mode is out "winda_display.txt";
50         -- plik wyjsciowy
51         variable linia : line; -- bufor lini
52         variable pietro : integer; -- pietra jako liczba
53     begin
54         if(clk'event and clk='1') then
55             pietro := conv_integer(floor_reg);
56             writeline(display, linia);
57             if stan = GO_UP then
58                 write(linia, string'("  ^  "));
59             elsif stan = GO_DOWN then
60                 write(linia, string'("  v  "));
61             elsif stan = ALARM_S then
62                 write(linia, string'(" ALARM "));
63             elsif stan = ERROR_S then
64                 write(linia, string'(" AWARIA"));
65             else
66                 write(linia, string'("      "));
67             end if;
68
69             write(linia, pietro);
70         end if;
71     end process wyswietlacz;
72
73     rejestry: process(clk, reset)
74     begin
75         if reset = '1' then
76             stan <= IDLE; -- Początkowy stan:
77             -- bezczynność
78             floor_reg <= (others => '0'); -- Początkowe pietro
79             -- parter (0)
80             req_reg <= (others => '0'); -- Brak zadan pieter
81             target_floor <= (others => '0'); -- Brak docelowego

```

```

79         pietra
move_cnt <= (others => '0'); -- Licznik ruchu
80         wyzerowany
door_timer <= (others => '0'); -- Licznik drzwi
81         wyzerowany
elsif (clk'event and clk = '1') then
82     stan <= next_stan; -- aktualizacja stanu
83
84     if stan /= ERROR_S then
85         req_reg <= req_reg or floor_req; -- zatrzymywanie
            zadanego pietra
86     end if;
87
88     -- Kasowanie zadania po otwarciu drzwi
89     if stan = OPEN_DOOR then
90         case floor_reg is
91             when "000" => req_reg(0) <= '0'; -- parter
92             when "001" => req_reg(1) <= '0'; -- 1 pietro
93             when "010" => req_reg(2) <= '0'; -- 2 pietro
94             when "011" => req_reg(3) <= '0'; -- 3 pietro
95             when "100" => req_reg(4) <= '0'; -- 4 pietro
96             when others => null;
97         end case;
98     end if;
99     -- Wybieranie zadanego pietra i priorytet pieter
100     if stan = DECIDE then
101         if req_reg(4) = '1' then target_floor <= "100"; --
            pietro 4
102             elsif req_reg(3) = '1' then target_floor <=
                "011";
103             elsif req_reg(2) = '1' then target_floor <=
                "010";
104             elsif req_reg(1) = '1' then target_floor <=
                "001";
105             elsif req_reg(0) = '1' then target_floor <=
                "000"; -- parter
106         end if;
107     end if;
108     -- Licznik przemieszczenia
109     if stan = GO_UP or stan = GO_DOWN then
110         if move_cnt = "100" then -- TAKT WCZESNIEJ
111             if stan = GO_UP then
112                 floor_reg <= floor_reg + 1;
113             else
114                 floor_reg <= floor_reg - 1;
115             end if;
116         end if;

```

```

117         if move_cnt = "101" then
118             move_cnt <= (others => '0');
119         else
120             move_cnt <= move_cnt + 1;
121         end if;
122     elsif stan = OPEN_DOOR or reset = '1' then
123         move_cnt <= (others => '0'); -- zeruj przy
124             otwarciu drzwi bacz resece
125     end if;
126
127     -- Licznik czasu otwarcia drzwi
128     if stan = WAIT_DOOR then
129         door_timer <= door_timer + 1;
130     else
131         door_timer <= (others => '0');
132     end if;
133 end if;
134 end process rejestry;
135
136 -- Logika przejść
137 process(stan, floor_reg, req_reg, target_floor, move_cnt,
138         door_timer, alarm, error, open_btn, close_btn)
139 begin
140     next_stan <= stan;      -- Domyslnie pozostan w aktualnym
141         stanie
142     motor_on <= '0';        -- Silnik wylaczony
143     motor_dir <= '0';        -- Kierunek: dol (nieistotny gdy
144         silnik wylaczony)
145     door_state <= '0';      -- Drzwi zamkniete
146     display <= '1';         -- Wswietlacz wlaczony
147     alarm_led <= '0';       -- LED alarmu wylaczony
148     error_led <= '0';       -- LED awarii wylaczony
149
150     case stan is
151
152         when IDLE => -- oczekuje w miejscu na zadania pieter lub
153             obslugi drzwi/awarii/alarmu
154             display <= '0';
155             if error = '1' then
156                 next_stan <= ERROR_S;
157             elsif alarm = '1' then
158                 next_stan <= ALARM_S;
159             elsif req_reg /= "00000" then
160                 next_stan <= DECIDE;
161             elsif open_btn = '1' then
162                 next_stan <= OPEN_DOOR;

```

```

159         end if;
160
161     when DECIDE => -- Decyzja o kierunku jazdy windy
162         if error = '1' then
163             next_stan <= ERROR_S;
164         elsif alarm = '1' then
165             next_stan <= ALARM_S;
166         else
167             if (floor_reg < "100" and req_reg(4) = '1') or --
168                 sprawdzanie czy mamy zadania powyzej
169                 aktualnego pietra
170                 (floor_reg < "011" and req_reg(3) = '1') or
171                 (floor_reg < "010" and req_reg(2) = '1') or
172                 (floor_reg < "001" and req_reg(1) = '1') then
173                 next_stan <= GO_UP; -- jazda do gory
174             elsif (floor_reg > "000" and req_reg(0) = '1') or
175                 -- sprawdzanie czy mamy zadania pieter ponizej
176                 aktualnego
177                 (floor_reg > "001" and req_reg(1) = '1')
178                 or
179                 (floor_reg > "010" and req_reg(2) = '1')
180                 or
181                 (floor_reg > "011" and req_reg(3) = '1')
182                 then
183                 next_stan <= GO_DOWN; -- jazda w dol
184             else
185                 next_stan <= OPEN_DOOR; -- jak jestesmy na
186                 tym pietrze to otworz drzwi
187             end if;
188         end if;
189
190     when GO_UP => -- stan obslugujacy jazde do gory
191         if error = '1' then
192             next_stan <= Error_S;
193         elsif alarm = '1' then
194             next_stan <= ALARM_S;
195         else
196             motor_on <= '1'; -- wlacz silnik
197             motor_dir <= '1'; -- kierunek gora
198
199             if move_cnt = "101" then -- sprawdz czy
200                 osiagnelismy pietra
201                 if (floor_reg = "001" and req_reg(1) = '1')
202                     or -- sprawdzanie czy na aktualnym
203                         pietrze bylo jakies zadanie jesli tak to
204                         otworz drzwi
205                     (floor_reg = "010" and req_reg(2) =

```



```

194         '1') or
        (floor_reg = "011" and req_reg(3) =
195         '1') or
        (floor_reg = "100" and req_reg(4) =
196         '1') then
        next_stan <= OPEN_DOOR;
197     end if;
198 end if;
199 end if;
200
201 when GO_DOWN =>
202     if error = '1' then
203         next_stan <= Error_S;
204     elsif alarm = '1' then
205         next_stan <= ALARM_S;
206     else
207         motor_on <= '1'; -- włącz silnik
208         motor_dir <= '0'; -- kierunek dol
209         if move_cnt = "101" then
210             if floor_reg = target_floor then -- sprawdź
211                 czy jest to piętro docelowe
212                 next_stan <= OPEN_DOOR;
213             elsif (floor_reg = "000" and req_reg(0) =
214                 '1') or -- sprawdzanie czy na aktualnym
215                 piętrze było jakieś zadanie jeśli tak to
216                 otwórz drzwi
217                 (floor_reg = "001" and req_reg(1) =
218                 '1') or
219                 (floor_reg = "010" and req_reg(2) =
220                 '1') or
221                 (floor_reg = "011" and req_reg(3) =
222                 '1') then
223                 next_stan <= OPEN_DOOR;
224             end if;
225         end if;
226     end if;
227 end if;
228
229 when OPEN_DOOR =>
230     if error = '1' then
231         next_stan <= Error_S;
232     elsif alarm = '1' then
233         next_stan <= ALARM_S;
234     else
235         door_state <= '1'; -- drzwi otwarte
236         next_stan <= WAIT_DOOR; -- przejście do
237             oczekiwania na zamknięcie
238     end if;

```

```

230
231     when WAIT_DOOR => -- drzwi otwarte oczekują na zamknięcie
                        guzikiem/automatycznie
232         if error = '1' then
233             next_stan <= Error_S;
234         elsif alarm = '1' then
235             next_stan <= ALARM_S;
236         else
237             door_state <= '1';
238             if close_btn = '1' then -- warunki zamknięcia
                                    drzwi
239                 next_stan <= CLOSE_DOOR;
240             elsif door_timer = "1001" then -- automatyczne
                                    zamknięcie
241                 next_stan <= CLOSE_DOOR;
242             elsif open_btn = '1' then -- przedłużenie
                                    zamykania drzwi
243                 next_stan <= OPEN_DOOR;
244             end if;
245         end if;
246
247     when CLOSE_DOOR => -- stan przejściowy decyzja czy idle czy
                        wybór kierunku
248         if error = '1' then
249             next_stan <= Error_S;
250         elsif alarm = '1' then
251             next_stan <= ALARM_S;
252         else
253             if req_reg /= "00000" then -- jeśli kolejne
                                    zadania to wybór kierunku
254                 next_stan <= DECIDE;
255             else
256                 next_stan <= IDLE; -- brak zadan to idle
257             end if;
258         end if;
259
260     when ALARM_S =>
261         alarm_led <= '1'; -- zapalenie LED
262         door_state <= '1'; -- otwarcie drzwi
263         motor_on <= '0'; -- wyłącz silnik
264         if alarm = '0' then -- jeśli alarm wyłączony to przejdź
                                do idle
265             next_stan <= IDLE;
266         end if;
267
268     when Error_S =>
269         error_led <= '1'; -- zapalenie led

```

```

270         motor_on <= '0'; -- wylacz silnik
271         if reset = '1' and error = '0' then -- oczekiwanie
272             na reset i usuniecie awarii.
273             next_stan <= Idle;
274         end if;
275
276         when others =>
277             next_stan <= IDLE;
278         end case;
279     end process;
280 end Behavioral;

```

Listing 1: Kod źródłowy sterownika windy (Winda.vhd)